



Project Title

**EXPLAINABLE AI-DRIVEN LIGHTWEIGHT NETWORK INTRUSION DETECTION
SYSTEM**

**Muhammed Bertuğhan KILINÇ 19COMP1102, Ayşe KORKUT 21COMP1011, Mert ÖZBAY
21COMP1023**

REQUIREMENTS ANALYSIS DOCUMENT

1. Introduction

The risk of cyberattacks has greatly increased due to the quick development of digital infrastructures and network-connected systems. To identify questionable activity and stop security incidents, organizations rely on dependable and secure network monitoring systems. Signature-based detection methods, which rely on previously identified attack patterns, are frequently the foundation of traditional intrusion detection systems (IDS). However, these systems may produce a lot of false positives and frequently fail to identify new or unknown attacks.

In cybersecurity research, machine learning-based anomaly detection techniques have drawn more attention as a solution to these limitations. These systems examine patterns in network traffic and identify anomalies.

The goal of this project is to create a lightweight, AI-based network anomaly detection system that can use machine learning techniques to analyze network traffic and identify unusual activity.

Network traffic will be recorded by the system, which will then extract key characteristics, analyze them using an anomaly detection model, and present the findings via a monitoring dashboard.

The suggested system aims to offer a straightforward, effective, and adaptable method of identifying questionable network activity.

1.1. Purpose of the System

The purpose of this project is to develop a lightweight anomaly detection platform that can use network traffic and detect unusual patterns using machine learning algorithms.

The system will assist security analysts in identifying suspicious activities by providing real-time monitoring and anomaly alerts.

The system is designed to support the following main functions:

- Capture network traffic

- Extract network features
- Analyze traffic using machine learning models
- Detect anomalies
- Generate alerts
- Provide a monitoring dashboard for analysts

Unlike traditional signature-based methods, this system aims to improve security monitoring by providing intelligent detection capabilities.

1.2. Scope of the System

This project involves the design and development of a machine learning-based anomaly detection system that analyzes network traffic data and identifies abnormal behavior.

The system will include the following components:

- Network traffic capturing module
- Feature extraction module
- Machine learning anomaly detection model
- Backend service for data processing
- Database for storing results
- Web-based monitoring dashboard

The system will focus on anomaly detection and visualization of network behavior.

The following components are outside the scope of this project:

- Real-time automated response mechanisms
- Integration with enterprise security infrastructure
- Large-scale distributed network monitoring systems
- Advanced threat intelligence integration

1.3. Objectives and Success Criteria of the Project

The main objective of the project is to design and implement a prototype anomaly detection system that demonstrates how machine learning techniques can be used to detect abnormal network activities.

The success of the project will be evaluated based on the following criteria:

- The system successfully captures network traffic data.
- Features can be extracted from network packets.
- The machine learning model can analyze network behavior.
- The system can generate anomaly alerts.
- The system operates reliably during test scenarios.

1.4. Definitions, Acronyms, and Abbreviations

IDS: Intrusion Detection System

ML: Machine Learning

AI: Artificial Intelligence

Anomaly Detection: Identifying unusual patterns in data that do not follow to expected behavior

Network Traffic: Data packets transmitted across a network

Alert: Notification generated when suspicious activity is detected

Dashboard: Graphical interface used to monitor system outputs

XAI: Explainable Artificial Intelligence

1.5. Overview

This document presents the requirements analysis for the proposed anomaly detection system. The document is organized as follows:

- Section 1 introduces the system and its objectives.
- Section 2 describes the current systems and their limitations.
- Section 3 presents the proposed system and its requirements.
- Section 4 discusses potential risks and project planning.

2. Current System

Currently, many organizations rely on traditional intrusion detection systems to monitor network activity. These systems are typically based on predefined signatures of known attacks.

While signature-based systems are effective at detecting known threats, they have several limitations. They are unable to detect previously unseen attack patterns and require frequent updates to remain effective.

Another common issue with traditional IDS systems is the high number of false positives, which can overwhelm security analysts and reduce the effectiveness of monitoring systems.

Several modern security solutions attempt to include machine learning techniques into network monitoring systems. However, these systems often require complex infrastructure and large computational resources.

Therefore, there is a need for a lightweight anomaly detection system that can provide intelligent detection capabilities while maintaining simplicity and efficiency.

3. Proposed System

The proposed system is an Explainable AI-Driven Lightweight Network Intrusion Detection System designed to analyze network traffic and identify abnormal behavior.

The system architecture consists of multiple components working together to capture data, process network traffic, analyze patterns using machine learning models, and present results through a

monitoring dashboard. The system includes some main modules like network traffic capturing module, feature extraction module, alert generation module and monitoring dashboard.

3.1. Overview

Presents a functional overview of the system. Various stakeholders, actors can be described in the overview.

The system will be used primarily by security analysts who monitor network activities and investigate anomalies.

The system captures network traffic data from the network interface and processes this data using a feature extraction module. The extracted features are passed to a machine learning model that calculates anomaly scores.

If the anomaly score exceeds a predefined threshold, the system generates an alert and stores it in the database.

The results are presented through a web-based dashboard where analysts can view alerts and investigate suspicious activities.

3.2. Requirements

Our proposed AI-Driven Lightweight Network Intrusion Detection System monitors the local network to find unusual activities. It does not use traditional, known attack rules. Instead, the system first captures raw network packets. Then, it converts these packets into features like packet count, byte rate, and entropy. After, an unsupervised machine learning model (Isolation Forest) checks these features and calculates an anomaly score for the network traffic.

One of the most important parts of the system is the Explainable AI (XAI) layer. We use SHAP analysis to show exactly which features caused the anomaly alert. Finally, a REST API sends these risk scores, alerts, and feature explanations to a web dashboard.

For security analysts, especially those in small companies, this system is a clear and useful tool. Analysts can see network problems in real time. They can also easily understand why the system gave a warning. This transparency makes our system much more reliable and easier to use than traditional "black-box" AI models.

3.2.1. Use Cases

UC-01: View Monitoring Dashboard

- **Goal:** The user wants to monitor real-time network traffic and see active threats.
- **Primary Actor:** Security Analyst
- **Priority:** High
- **Main Success Scenario:**

1. **User:** Logs into the web-based monitoring dashboard.
2. **System:** Connects to the backend database and retrieves the latest network statistics.
3. **System:** Displays real-time traffic data, recent anomaly scores, and active security alerts on the screen.
4. **User:** Reviews the list to see if any high-risk alerts need attention.

UC-02: Investigate Alert & View AI Explanation(SHAP)

- **Goal:** The user wants to inspect a specific security alert and understand why the AI flagged it as an anomaly.
- **Primary Actor:** Security Analyst
- **Priority:** High
- **Main Success Scenario:**
 1. **User:** Clicks on a high-risk security alert from the dashboard list.
 2. **System:** Opens a detailed view showing the suspicious IP address and network flow features.
 3. **User:** Clicks the "Explain Decision" button to view the SHAP analysis.
 4. **System:** Generates and displays a visual chart highlighting which specific network features caused the high anomaly score.
 5. **User:** Uses this explanation to verify the threat and decide on a response.

UC-03: Train Machine Learning Model

- **Goal:** The user wants to train the anomaly detection model with a new dataset to improve its accuracy.
- **Primary Actor:** Security Analyst (or System Administrator)
- **Priority:** Medium
- **Main Success Scenario:**
 1. **User:** Navigates to the "Model Management" section of the system.
 2. **User:** Uploads a new dataset (e.g., normal network behavior logs).
 3. **User:** Initiates the training process for the Isolation Forest model.
 4. **System:** Processes the dataset, updates the model parameters, and displays a "Training Complete" confirmation along with performance metrics.

UC-04: Manage System Configuration

- **Goal:** The user needs to adjust monitoring settings and threat detection thresholds.
- **Primary Actor:** System Administrator
- **Priority:** Medium
- **Main Success Scenario:**

1. **User:** Accesses the "Settings" menu.
2. **System:** Displays current configurations, including alert thresholds and network interfaces being monitored.
3. **User:** Updates the anomaly score threshold to reduce false positives.
4. **User:** Clicks the "Save Configuration" button.
5. **System:** Applies new settings to the running monitoring module and shows a successful message.

UC-05: Export Threat Logs

- **Goal:** The user wants to download the history of detected anomalies for reporting or further analysis.
- **Primary Actor:** Security Analyst
- **Priority:** Medium
- **Main Success Scenario:**
 1. **User:** Navigates to the "Logs" or "History" section on the dashboard.
 2. **System:** Displays the historical detection results stored in the database.
 3. **User:** Selects a specific date range and clicks the "Export Logs" button.
 4. **System:** Compiles the requested data into a structured file format (e.g., CSV).
 5. **System:** Prompts the user to download the file.

3.2.2. Scenarios

This section details specific instances of our high-priority Use Cases. The scenarios outline the step-by-step interactions between the primary actor and the system, including preconditions, normal flows, and possible exceptions.

Scenario 1: Investigate Alert & View AI Explanation (Instance of UC-02)

- **Primary Actor:** Security Analyst
- **Trigger:** The analyst notices a high-risk security alert on the dashboard.
- **Preconditions:**
 - The monitoring system is active and capturing traffic.
 - The user is securely logged into the dashboard.
- **Normal Flow:**
 1. System displays an active anomaly alert with a high risk score.
 2. Analyst clicks on the specific alert for detailed analysis.
 3. System opens a detailed view showing the suspicious IP address and flow statistics.
 4. Analyst clicks the "Explain Decision" button to request AI transparency.
 5. System retrieves the SHAP analysis data for that specific traffic instance.

6. System generates and displays a visual chart highlighting the exact features (e.g., unusual byte rate) that caused the anomaly score.
 7. Analyst uses the visual explanation to verify the threat and blocks the IP address.
- **Postconditions:**
 - Threat is verified and mitigated.
 - Alert status is updated in the database.
 - **Exceptions:**
 - E1 – Insufficient Data for Explanation: If the system cannot generate a reliable SHAP chart due to incomplete packet data, it displays a warning: "Explanation unavailable: Insufficient feature data." The analyst must rely on basic flow statistics.

Scenario 2: Train Machine Learning Model (Instance of UC-03)

- **Primary Actor:** Security Analyst / System Administrator
- **Trigger:** The model requires updating with recent normal network behavior data.
- **Preconditions:**
 - A formatted dataset (e.g., CSV) is available on the local machine.
 - The system is running in maintenance or training mode.
- **Normal Flow:**
 1. Analyst navigates the "Model Management" section.
 2. Analyst uploads the new dataset.
 3. System validates the dataset format and checks for required feature columns.
 4. Analysts initiate the training process for the Isolation Forest model.
 5. System trains the model and evaluates its performance metrics.
 6. System displays a "Training Complete" confirmation.
 7. System saves the updated model for future anomaly detection.
- **Postconditions:**
 - A newly trained Isolation Forest model is available and active in the system.
- **Exceptions:**
 - E2 – Invalid Dataset Format: If the uploaded dataset is missing crucial feature columns, the system halts the process and displays an error: "Training Failed: Invalid dataset format." The model is not updated.

Scenario 3: Handle a False Positive Alert

- **Primary Actor:** Security Analyst
- **Trigger:** The system flags an unusual but legitimate network activity.

- **Preconditions:**

- Monitoring system is active.

- **Normal Flow:**

1. Analyst opens the dashboard and selects a medium-risk alert.
2. Analyst clicks "Explain Decision" to view the SHAP analysis.
3. System displays the explanation chart.
4. Analyst reviews the chart and realizes the "anomaly" was caused by a known, safe internal file transfer (expected high byte rate).
5. Analyst clicks "Mark as False Positive".
6. System updates the alert status and logs the feedback to help tune future thresholds.

- **Postconditions:**

- The alert is dismissed without blocking the IP address.

3.3. Nonfunctional Requirements

This section defines the quality attributes, constraints, and performance goals of the proposed AI-Driven Lightweight Network Intrusion Detection System.

Usability

The system is designed for security analysts, particularly those in small and medium-sized organizations who might not have deep expertise in machine learning. Thanks to the Explainable AI (SHAP) integration, the system does not just show a warning; it visually explains why a specific traffic flow is suspicious. This makes the dashboard highly intuitive and easy to use for decision-making.

Reliability

Since the system monitors network traffic, it must run continuously. If the machine learning module or the backend API encounters an error, the system should log into the failure and attempt to restart the monitoring process without freezing the whole server. The anomaly detection model must also maintain an acceptable false positive rate to ensure analysts do not ignore the alerts.

Performance

This is a "lightweight" system, performance is a critical requirement.

- The system must have low inference latency, meaning the time between capturing a packet and generating an anomaly score should be minimal.
- It must consume low CPU and memory resources, ensuring it can run efficiently on standard local servers or small-scale hardware without needing expensive enterprise-grade equipment.

Supportability and Maintainability

The system uses modular architecture. The data processing, machine learning, and visualization layers are separated. This means future developers can update the machine learning model or change the dashboard interface without breaking the entire system.

Implementation

The system will be implemented to run on a local server environment. It relies on open-source tools and libraries, utilizing Python for the core logic, Scikit-learn for the Isolation Forest model, and FastAPI for the backend services.

External Interface

The system exposes a RESTful web service via FastAPI. While the primary goal is to feed data to our own web interface, this API acts as a programmatic external interface. It allows potential future integrations where other local security tools could automatically fetch anomaly scores or threat logs from our system.

Legal, Ethical, and Social Constraints

- **Legal & Privacy:** The system captures network traffic. To comply with data privacy laws (like GDPR or KVKK), the feature extraction pipeline will focus on flow-based behavioral statistics (like packet counts, byte rates, and port distributions) rather than deeply inspecting or logging sensitive user payload data.
- **Social Impact:** The growing complexity of cyber threats makes it hard for smaller organizations to defend themselves. Enterprise-grade IDS platforms are often too expensive and complex. This project addresses a community need by providing an accessible, transparent, and lightweight security solution that improves trust in AI-based security tools.

3.4. System Models

3.4.1. Use Case Model

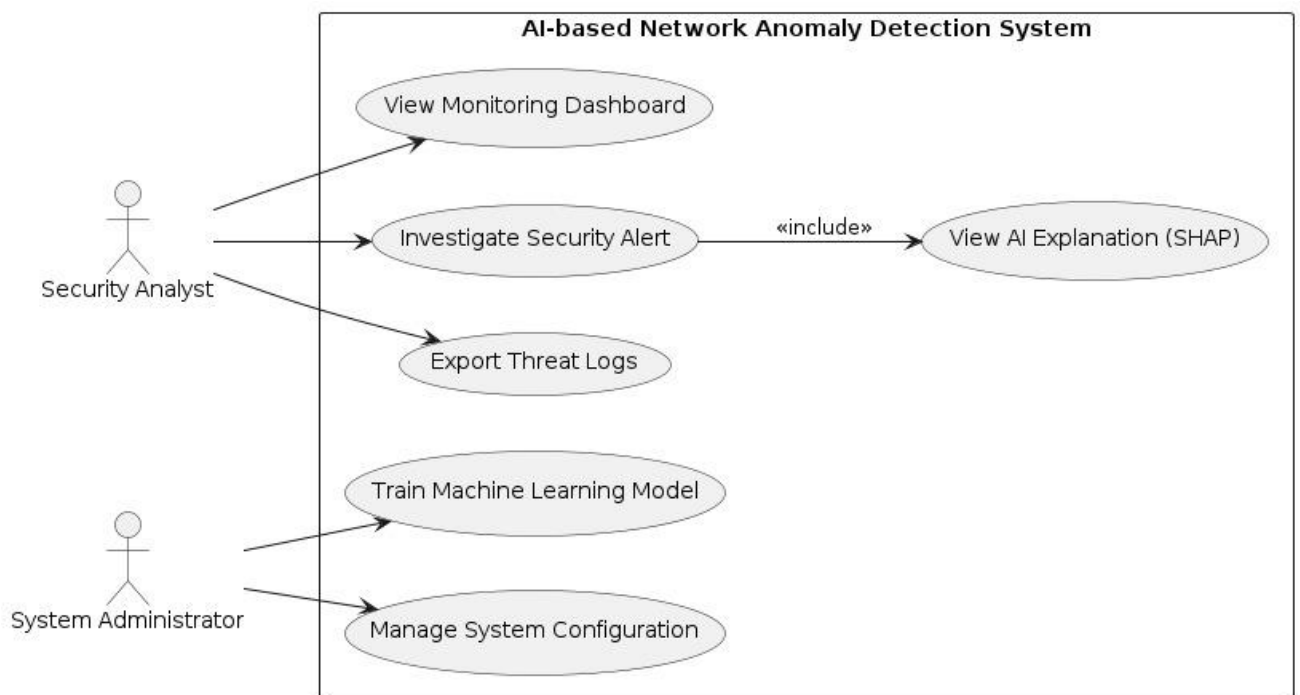


Figure 1: Use Case Model of System

3.4.2. Object Model

The analysis object model acts as a visual dictionary for the AI-Driven Lightweight Network Intrusion Detection System. It represents the main concepts and modules visible to the user and the system's core architecture. The UML class diagram includes the following main classes, along with their key attributes and operations:

1. NetworkFlow This class represents the captured and processed network traffic data.

- **Attributes:** sourceIP, destinationIP, packetCount, byteRate, flowDuration, protocolType.
- **Operations:** extractFeatures(), getFlowStatistics().

2. AnomalyDetector This class represents the unsupervised machine learning model, specifically the Isolation Forest algorithm.

- **Attributes:** modelStatus, anomalyThreshold.
- **Operations:** trainModel(), calculateAnomalyScore().

3. XAI_Engine This class is responsible for the Explainable AI layer using SHAP analysis.

- **Attributes:** explainerType, featureImportanceValues.
- **Operations:** generateExplanation(), highlightInfluentialFeatures().

4. SecurityAlert This class represents the warnings generated by the system and shown on the visualization dashboard.

- **Attributes:** alertID, timestamp, anomalyScore, threatSeverity, status.
- **Operations:** triggerAlert(), updateAlertStatus().

5. DashboardUser

This class represents the security analyst interacting with the web interface.

- **Attributes:** username, role.

- **Operations:** viewAlerts(), viewSHAPExplanations(), exportLogs().

AI-Driven Lightweight Network Intrusion Detection System - Analysis Object Model

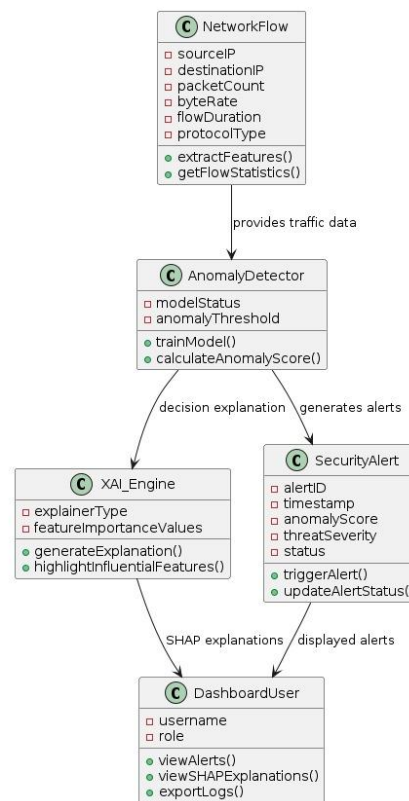


Figure 2: UML Diagram of System

3.4.3. Dynamic Model

This section illustrates the behavioral aspects of the AI-Driven Lightweight Network Intrusion Detection System. It shows how different user-level concepts and objects interact over time. We use a sequence diagram to model the interactions during our most critical use case, and a state machine to track the lifecycle of a security alert.

Sequence Diagram: Investigate Alert and View AI Explanation

The sequence diagram models the interactions that occur when a Security Analyst investigates a detected anomaly.

- The process begins when the analyst selects an active alert on the Monitoring Dashboard.
- The Dashboard requests the anomaly details from the Detection Engine.
- To understand the AI's decision, the analyst clicks "Explain Decision".
- The Dashboard sends an explanation request to the XAI Engine.
- The XAI Engine retrieves the flow-based features and calculates the SHAP values to determine feature importance.
- Finally, the XAI Engine returns the visual explanation chart back to the Dashboard, which displays it to the Analyst.

Alert Investigation and XAI Explanation Sequence

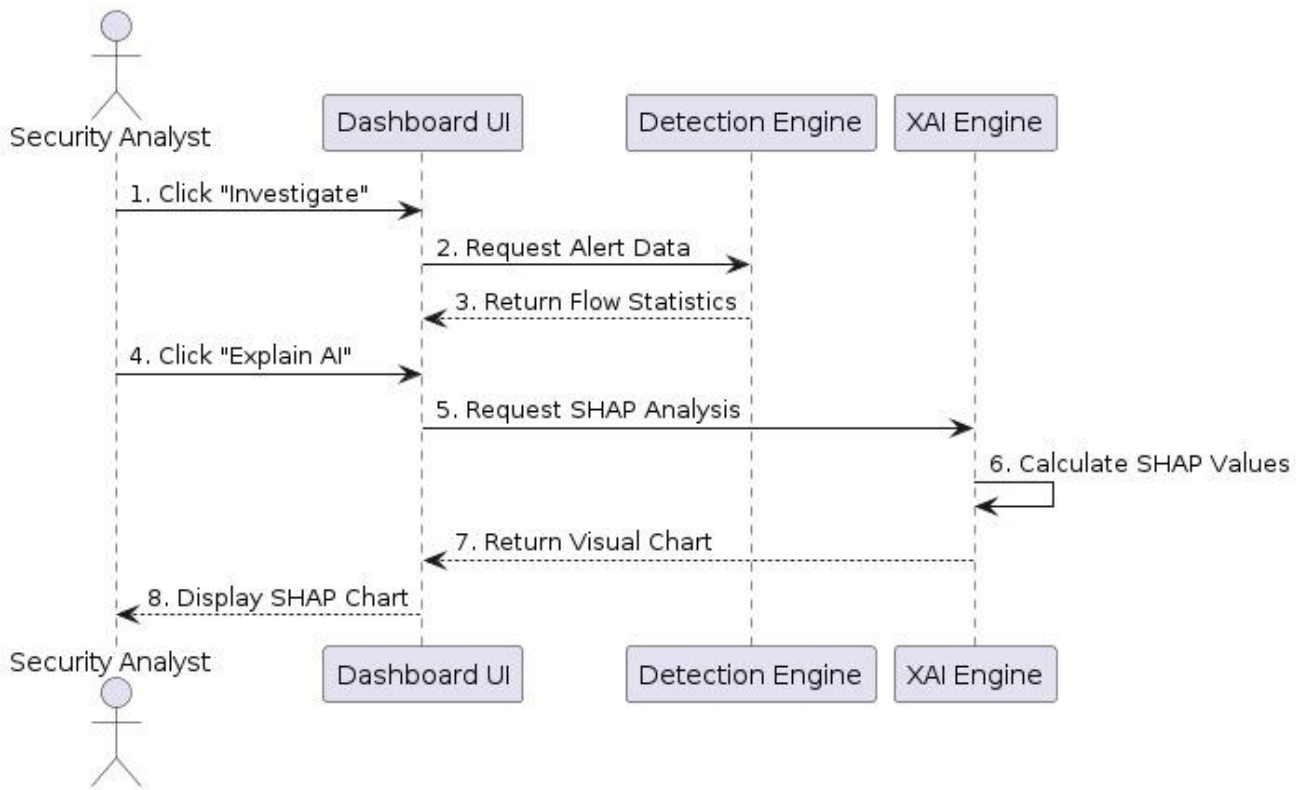


Figure 3: Sequence Diagram of Alert Investigation and XAI Explanation

State Machine: Security Alert Lifecycle

The state machine diagram represents the behavior and status changes of a single "Security Alert" object within the system.

- **New:** The alert is created when the Detection Engine assigns a high anomaly score.
- **Displayed:** The alert is pushed to the dashboard and waits for user action.
- **Investigating:** The Security Analyst clicks on the alert to view the SHAP explanations.
- From the "Investigating" state, the alert can move to two final states:
 - **Mitigated/Resolved:** The analyst confirms the threat and blocks the IP.
 - **False Positive:** The analyst determines the traffic is safe and dismisses the alert.

Security Alert Lifecycle - State Machine Diagram

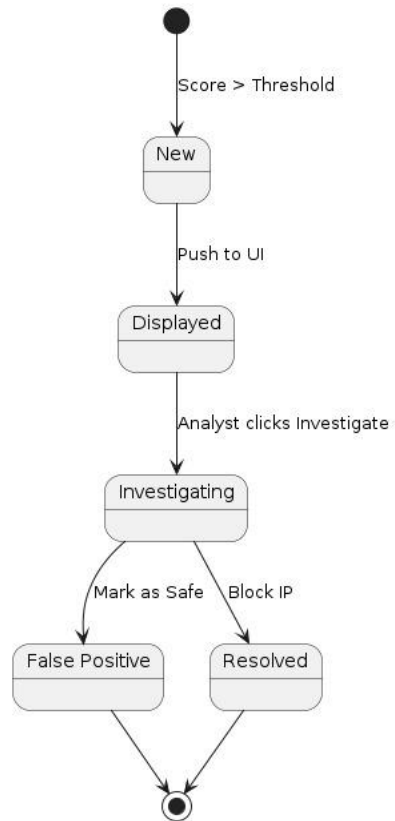


Figure 4: State Machine Diagram for Security Alert Lifecycle

3.4.4. User Interface—Navigational Paths and Screen Mock-ups

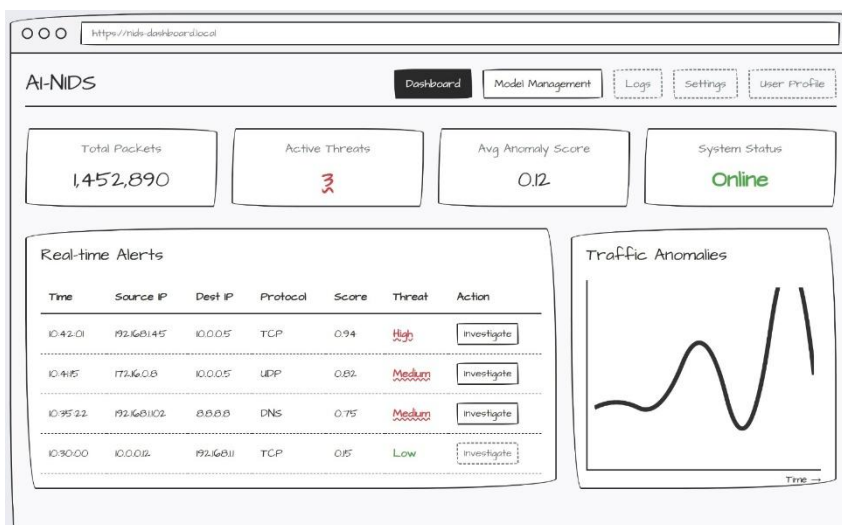


Figure 5: UI for Web- Based Dashboard 1

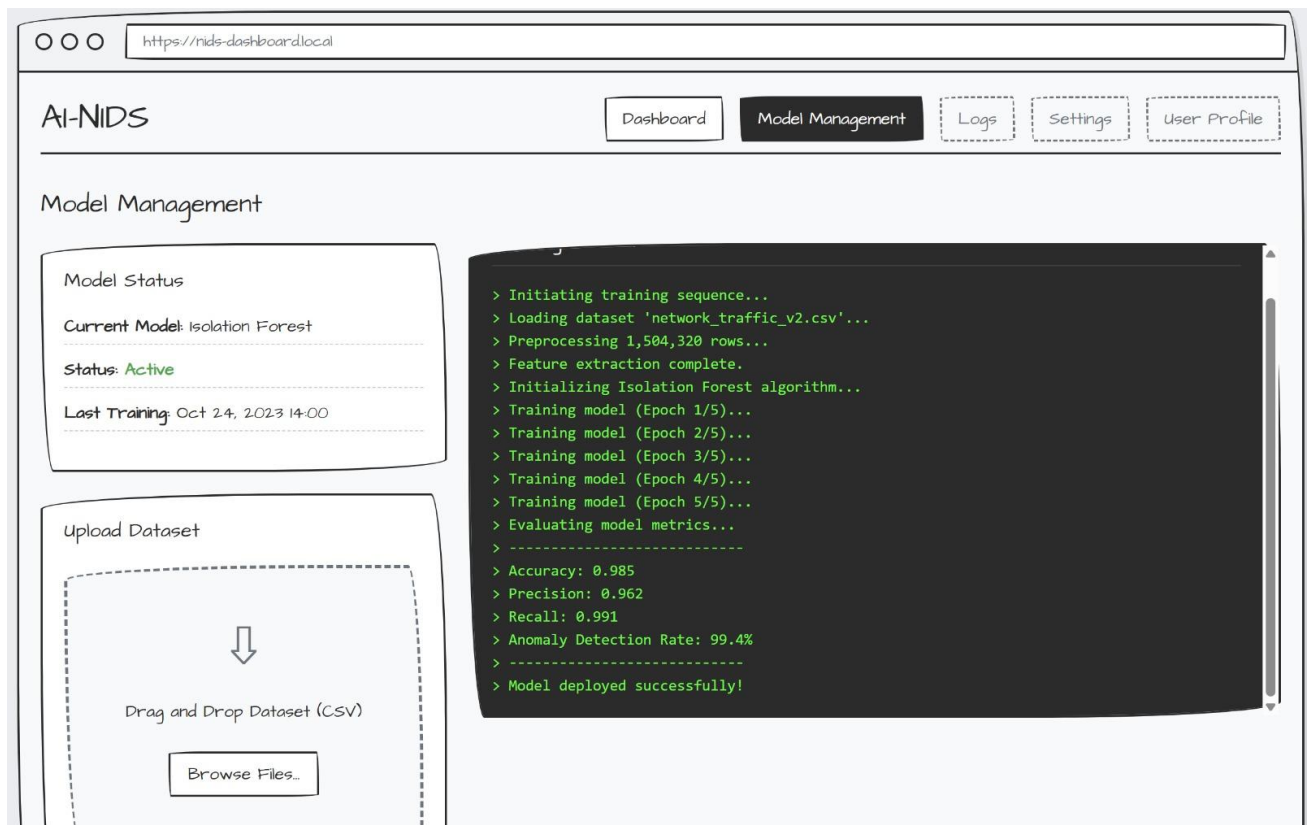


Figure 6: UI for Web- Based Dashboard 2

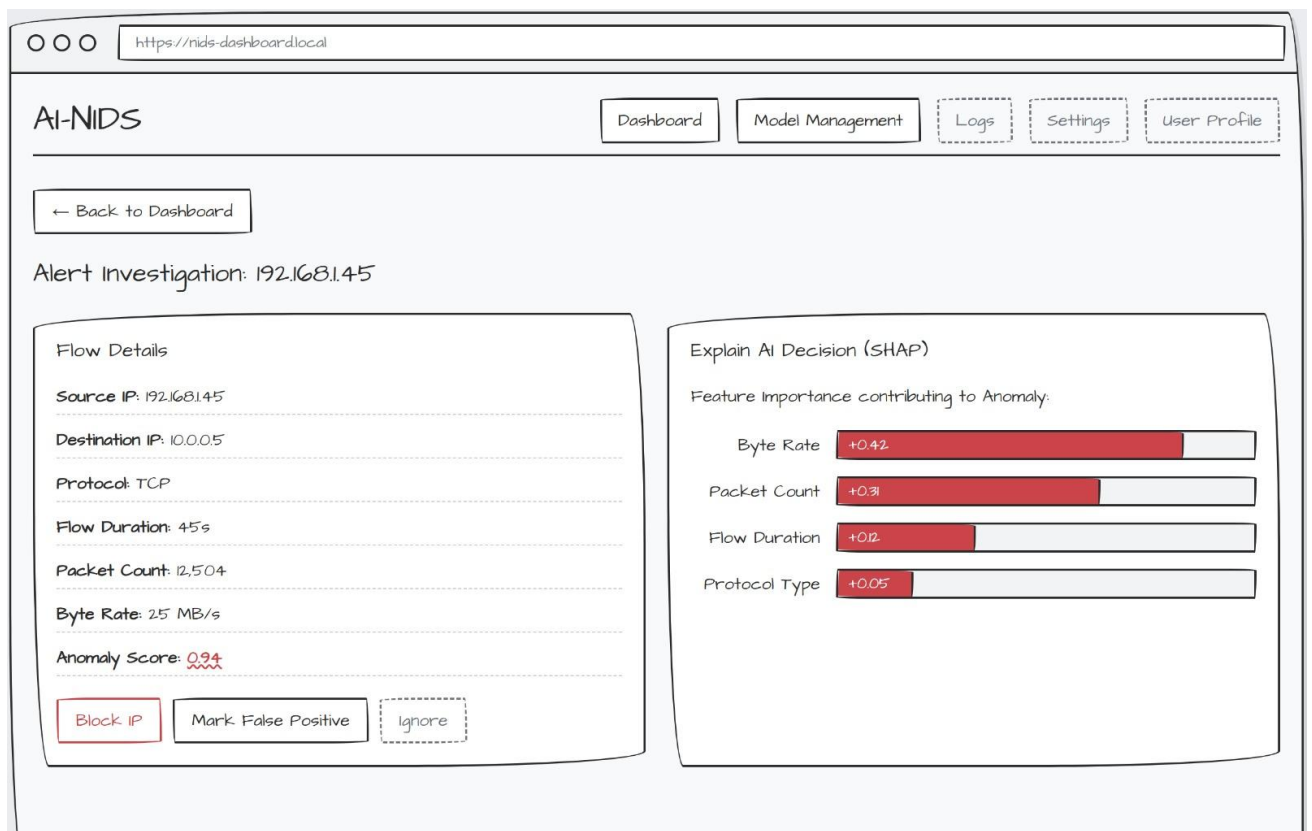


Figure 7: UI for Web- Based Dashboard 3

4. Other Analysis Elements

4.1.Risks and Alternatives

During the development of the AI-Driven Lightweight Network Intrusion Detection System, we might face several technical and schedule-related challenges. Table 2 outlines these potential risks, their likelihood, their impact on the project, and our alternative plans to mitigate them.

Table 2. Risks

	Likelihood	Effect on the project	B Plan Summary
High False Positive Rate (The ML model generates too many incorrect alerts)	Medium	High	Instead of Isolation Forest, we will test other unsupervised lightweight algorithms like Local Outlier Factor (LOF) or improve our feature extraction pipeline.
SHAP Analysis Latency (Explainable AI calculations slow down the real-time system)	Medium	High	We will compute SHAP values in the background (asynchronously) or limit the AI explanation feature to only "High-Risk" alerts to maintain a lightweight system.
Live Traffic Capture Issues (Permission or hardware problems while capturing local packets)	Low	Medium	We will use publicly available, pre-recorded network datasets (like CICIDS2017) and play them back to simulate a live traffic stream for the prototype demonstration.
Module Integration Delays (Backend, ML, and Frontend taking too long to connect)	Medium	Medium	We will use mock data (fake API responses) so the UI dashboard and ML engine can be developed independently at the same time. We will dynamically reassign tasks among the three team members if needed.

4.2.Project Plan

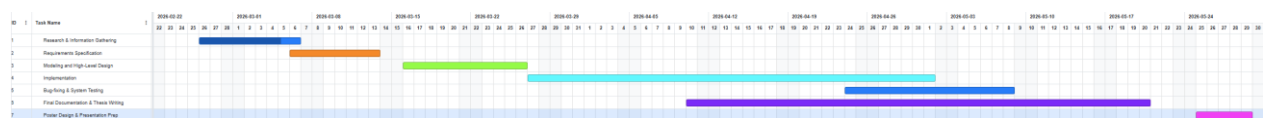


Figure 8: Gantt Chart

Table 3. List of work packages

WP#	Work package title	Members involved
WP1	Research & Information Gathering: Literature review on XAI, anomaly detection, and dataset selection.	All Members (Bertuğhan, Ayşe, Mert)
WP2	Requirements Specification: Defining functional, non-functional requirements, and use cases (Writing the RAD).	All Members (Bertuğhan, Ayşe, Mert)
WP3	Modeling and High-Level Design: Creating UML class diagrams, sequence diagrams, and UI dashboard mock-ups.	All Members (Bertuğhan, Ayşe, Mert)
WP4	System Implementation: Developing the core system. - <i>Network & Data Pipeline:</i> Mert ÖZBAY - <i>ML & XAI Engine:</i> Muhammed Bertuğhan KILINÇ - <i>Backend & Dashboard:</i> Ayşe KORKUT	All Members (Bertuğhan, Ayşe, Mert)
WP5	Bug-fixing & System Testing: Integrating the modules, resolving errors, and testing false positive rates.	All Members (Bertuğhan, Ayşe, Mert)
WP6	Final Documentation & Thesis Writing: Preparing the final technical report and formatting the project thesis.	All Members (Bertuğhan, Ayşe, Mert)
WP7	Poster Design & Presentation Prep: Creating the final presentation and preparing the live prototype demo.	All Members (Bertuğhan, Ayşe, Mert)

Table 4. Work Package 1

WP 1: Research & Information Gathering			
Start date: 2026-02-26 End date: 2026-03-06			
Leader:	Muhammed Bertuğhan KILINÇ	Members involved:	Ayşe KORKUT, Mert ÖZBAY
Objectives: <i>The main objective of this work package is to build the theoretical foundation of the project. The team will research Explainable AI (XAI) concepts, unsupervised anomaly detection algorithms (like Isolation Forest), and analyze public network datasets to find the most suitable one for the system.</i>			
Tasks: Task 1.1 Literature Review: <i>Review academic papers on lightweight IDS and SHAP analysis.</i> Task 1.2 Algorithm Selection: <i>Compare machine learning algorithms to finalize the use of Isolation Forest.</i> Task 1.2 Dataset Analysis: <i>Evaluate public datasets (such as CICIDS2017 or UNSW-NB15) for training purposes.</i>			

Deliverables

D1.1: Literature Review Summary.

D1.2: Finalized training dataset.

WP 2: Requirements Specification

Start date: 2026-03-06 **End date:** 2026-03-14

Leader:	Ayşe KORKUT	Members involved:	Muhammed Bertuğhan KILINÇ, Mert ÖZBAY
----------------	-------------	--------------------------	---------------------------------------

Objectives: This package focuses on defining the exact behavior and constraints of the system. The team will list all functional and non-functional requirements, write detailed use cases, and identify potential project risks to create the Requirements Analysis Document (RAD).

Tasks:

Task 2.1 Define System Requirements: List all functional and non-functional requirements (e.g., lightweight performance).

Task 2.2 Write Use Cases: Detail the interactions between the Security Analyst and the system (e.g., View AI Explanation).

Task 2.3 Risk Assessment: Identify potential risks and create alternative B-plans.

Deliverables

D2.1: Draft of the Requirements Analysis Document (RAD).

WP 3: Modeling and High-Level Design

Start date: 2026-03-15 **End date:** 2026-03-28

Leader:	Mert ÖZBAY	Members involved:	Muhammed Bertuğhan KILINÇ, Ayşe KORKUT
----------------	------------	--------------------------	--

Objectives: The goal is to visually design the software architecture before coding begins. The team will create UML diagrams to show how system components interact and design the user interface (UI) mock-ups for the monitoring dashboard.

Tasks:

Task 3.1 Object Modeling : Draw UML Class Diagrams for core components (TrafficCapture, ML_Engine, etc.).

Task 3.2 Dynamic Modeling: Create Sequence and State Machine diagrams for alert handling.

Task 3.3 UI Prototyping: Design screen mock-ups for the main dashboard and the SHAP explanation view.

Deliverables

D3.1: System Architecture Diagrams (UML).

D3.2: Dashboard UI Mock-ups.

D3.3: Finalized Requirements Analysis Document.

WP 4: Implementation (System Development)			
Start date: 2026-03-29 End date: 2026-05-05			
Leader:	Muhammed Bertuğhan KILINÇ	Members involved:	Ayşe KORKUT, Mert ÖZBAY
Objectives: This is the core development phase where the actual coding happens. The workload is divided into specific modules: building the data capture pipeline, training the machine learning model with XAI integration, and developing the backend API and frontend dashboard.			
Tasks: Task 4.1 Data Pipeline Development : Write scripts to capture packets and extract flow features. Task 4.2 ML and XAI Integration : Train the Isolation Forest model and implement SHAP logic for explanations. Task 4.3 Web Dashboard Development : Build the FastAPI backend and the web-based user interface.			
Deliverables D4.1: Working Feature Extraction Module. D4.2: Trained Anomaly Detection Model. D4.3: Functional Web Dashboard and API.			

WP 5: Bug-fixing & System Testing			
Start date: 2026-04-19 End date: 2026-05-14			
Leader:	Ayşe KORKUT	Members involved:	Muhammed Bertuğhan KILINÇ, Mert ÖZBAY
Objectives: This package ensures the developed system works reliably and efficiently. The team will combine all modules, test the system against simulated network attacks, tune the model to reduce false positives, and fix any software bugs.			
Tasks: Task 5.1 System Integration: Connect the data pipeline, ML engine, and web dashboard. Task 5.2 Performance Testing: Measure CPU/Memory usage and inference latency to ensure the system is lightweight. Task 5.3 Detection Evaluation: Calculate precision, recall, and false positive rates using test data.			
Deliverables D5.1: Software Test and Performance Report. D5.2: Stable and Integrated System Prototype.			

WP 6: Final Documentation & Thesis Writing			
Start date: 2026-04-12End date: 2026-05-24			
Leader:	<i>Mert ÖZBAY</i>	Members involved:	<i>Muhammed Bertuğhan KILINÇ, Ayşe KORKUT</i>
Objectives: <i>The objective is to compile all the project work into an academic format. The team will document the methodology, experimental results, system architecture, and conclusions into the final graduation thesis report.</i>			
Tasks: Task 6.1 Draft Methodology: <i>Write the technical sections explaining how the ML model and XAI work.</i> Task 6.2 Compile Results: <i>Add performance metrics, ROC curves, and SHAP visual examples to the document.</i> Task 6.3 Final Review: <i>Check the document for academic formatting and grammatical correctness.</i>			
Deliverables D6.1: <i>Completed Graduation Project Thesis/Report.</i>			

5. Glossary

To establish clear terminology and ensure this document is easily understood, the domain-specific technical terms and acronyms used throughout the text are defined below in alphabetical order:

- **Anomaly:** A deviation from normal network behavior that may indicate a security threat, unauthorized access, or an attack.
- **Anomaly Detection:** A machine learning approach that identifies unusual patterns that do not conform to expected network behavior, rather than looking for specific, known attack signatures.
- **False Positive:** A situation where the system incorrectly generates a security alert for normal, safe network traffic.
- **FastAPI:** A modern, high-performance web framework for building APIs with Python. It is used in this project to connect the machine learning backend with the web dashboard.
- **Feature Extraction:** The process of transforming raw network packets into structured, measurable data points (such as packet size, byte rate, or connection duration) that the machine learning model can process.
- **IDS (Intrusion Detection System):** A software application or system that continuously monitors network traffic for malicious activities or policy violations.
- **Isolation Forest:** An unsupervised machine learning algorithm used in this project. It detects anomalies by isolating abnormal data points, making it highly efficient for processing large amounts of data.
- **Lightweight:** A system design philosophy meaning the software consumes very low computational resources (CPU and RAM), allowing it to run effectively on standard computers rather than expensive, heavy enterprise servers.
- **Network Flow:** A sequence of network packets sent from a specific source to a specific destination during a single communication session.
- **REST API:** A software interface that allows different parts of the system (like the user interface and the database) to communicate and share data smoothly over the web.

- **SHAP (SHapley Additive exPlanations):** A mathematical method used to explain the output of machine learning models. It calculates and visualizes exactly how much each network feature contributed to an anomaly score.
- **XAI (Explainable Artificial Intelligence):** Artificial intelligence systems designed to be transparent. Unlike "black-box" AI, XAI allows human users (like security analysts) to understand, trace, and trust the reasons behind the AI's decisions.
- **Zero-Day Attack:** A completely new cyber threat or attack method that has never been seen before, meaning traditional security systems do not have a rule to stop it.

6. References

- [1] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in 2008 IEEE International Conference on Data Mining, Pisa, Italy, 2008, pp. 413–422.
- [2] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Advances in Neural Information Processing Systems (NeurIPS), 2017.
- [3] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in Proceedings of the International Conference on Information Systems Security and Privacy (ICISSP), 2018.
- [4] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems," in 2015 Military Communications and Information Systems Conference (MilCIS), 2015.
- [5] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in 2010 IEEE Symposium on Security and Privacy, pp. 305–316.
- [6] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," ACM SIGMOD Record, vol. 29, no. 2, pp. 93–104, 2000.
- [7] Scikit-learn Developers, "Isolation Forest," [Online]. Available: <https://scikit-learn.org> . [Accessed: Feb. 2026].
- [8] Zeek Network Security Monitor, "Zeek Documentation," [Online]. Available: <https://zeek.org> . [Accessed: Feb. 2026].
- [9] S. Lundberg, "SHAP Documentation," [Online]. Available: <https://shap.readthedocs.io> . [Accessed: Feb. 2026].